

# Management Service — Mimari Tasarım (SPEC)

**İş akışı aşaması:** 1/6 — SPEC (AGENTS.md "Geliştirme İş Akışı — 6 aşama"). **Hedef Kitle:** Geliştirici + saha mühendisi. **Amaç:** "Şu koşul sağlandığında şu komutu otomatik çalıştır, sonucu imzalı log'a bas, gerekirse alarm bildirimi gönder" otomasyonunu konfigüre edilebilir bir servisle sağlamak. Bugün kuralları biz (geliştirici) yazıyoruz; yarın aynı şema son kullanıcı arayüzünden (editör) üretilecek — mevcut manevra/komut konfigürasyon felsefesiyle aynı evrim.

## İçindekiler

1. Kapsam ve Hedef
2. Mevcut Altyapı ile İlişki
3. Bileşenler
4. Cycle Veri Toplama
5. Kural Modeli (Konfigürasyon Sözleşmesi)
6. Değerlendirme Semantiği (Davranış Sözleşmesi)
7. Aksiyonlar
8. Hata Kategorileri
9. Dağıtım ve Tier Wiring
10. Kabul Kriterleri
11. Görev Listesi (T1-T9)
12. Aşama Eşlemesi (6 aşamalı iş akışı)

## 1. Kapsam ve Hedef

**Problem:** Sistemde komutlar (tek cihaz, register yazma) ve manevralar (çok cihazlı komut zincirleri) mevcuttur ama hepsi **insan tetikli** çalışır. "BSC SoC %95'i geçerse şarjı durdur" gibi **veri tetikli** otomasyon yoktur.

### Kapsam (MVP):

- Cihaz telemetrisi üzerinde **kural tabanlı** koşul değerlendirme.
- Koşul sağlandığında **aksiyonlar**: cihaz komutu çalıştırma, imzalı log, alarm bildirimi.
- Kurallar **JSON konfigürasyon dosyasından** okunur (bugün biz yazarız).
- **queue\_management** kuyruğunun **ilk tüketicisi** olma.

### Kapsam dışı (Faz 1 — bilinçli erteleme):

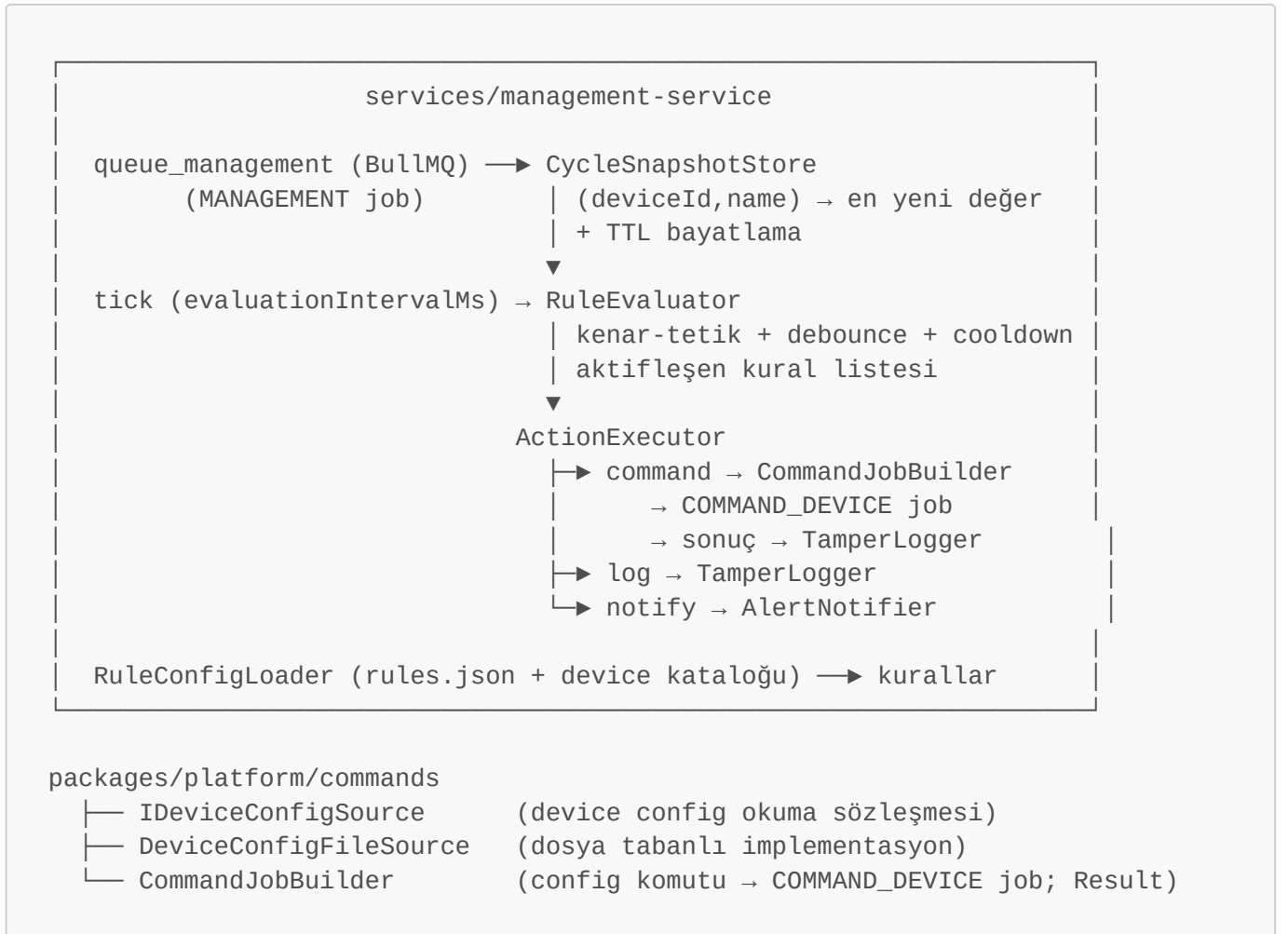
- Tam manevra zinciri aksiyonu (manevra kataloğu backend'e taşınınca).
- Kuralların DB + CRUD API + editör UI ile yönetimi.
- Expression engine (hesaplanmış metrikler, formül — v2-gap F1).
- Otomatik zaman senkronizasyonu (TEİAŞ #19) — management job'ına planlı, ayrı görev.

## 2. Mevcut Altyapı ile İlişki

| Mevcut yapı                                                         | Kullanım                                                                                         |
|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| MANAGEMENT job tipi + queue_management (shared-types/src/job.ts:60, | Veri kaynağı — device-service her okuma cycle'ında cihaz başına MANAGEMENT job yayınlar (device- |

| Mevcut yapı                                                               | Kullanım                                                                                                         |
|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>platform/messaging)</code>                                          | <code>scheduler.ts:44-82</code> ). Tüketici bugüne kadar yoktu.                                                  |
| <code>AlarmTransitionDetector</code> (device-service)                     | Desen kaynağı — kenar-tetikli dedup, yalnızca geçişlerde log. RuleEvaluator aynı felsefeyi kullanır.             |
| <code>AlertNotifier</code> (tamper-logger/src/sinks/alert-notifier.ts)    | Bildirim aksiyonu — eventCode başına cooldown (5 dk varsayılan).                                                 |
| <code>TamperLogger</code>                                                 | Aksiyon sonuç logları — imzalı zincir ( <code>auto_rule_* eventCode</code> 'ları).                               |
| <code>command-routes.ts</code> komut çözümleme (web-service, satır 32-85) | <code>CommandJobBuilder</code> olarak <code>packages/platform/commands</code> 'a çıkarılır; iki servis paylaşır. |
| <code>ISnapshotSource</code> deseni (ws-tunnel/src/snapshot/interface.ts) | CycleSnapshotStore aynı "en güncel değer" desenini kullanır.                                                     |
| Device config <code>type</code> alanı (üst seviye, zorunlu)               | <code>deviceTypes</code> koşul filtresinin çözülmesi.                                                            |

### 3. Bileşenler



| Bileşen                               | Sorumluluk                                                                                                                                                        | Bağımlılık (constructor)                                        |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| CycleSnapshotStore                    | MANAGEMENT job'larını kaydet; (deviceId, name) başına en yeni değeri tut; TTL geçmiş bayat say                                                                    | maxAgeMs, now( )                                                |
| RuleEvaluator                         | Snapshot üzerinde kuralları değerlendir; kenar-tetik + debounce + cooldown dedup; yalnızca aktifleşen kuralları döner                                             | now( )                                                          |
| ActionExecutor                        | Aksiyonları sırayla çalıştır; her aksiyon sonucunu logla; hata kademeli bozulma (aksiyon hatası kuralı değil, sonraki aksiyonları etkilemez — allSettled benzeri) | CommandJobBuilder, IMessageQueue, TamperLogger?, AlertNotifier? |
| RuleConfigLoader                      | rules.json yükle + zod doğrula + device katalogu kur (deviceId → type)                                                                                            | configDir                                                       |
| DeviceCatalog                         | Device config dosyalarından (deviceId → type) haritası; types filtresini cihaz setine çözümler                                                                    | IDeviceConfigSource                                             |
| ManagementService                     | Wiring + yaşam döngüsü: worker kaydı, tick zamanlayıcı, start/stop/health                                                                                         | tüm yukarıdakiler + ManagementServiceConfig                     |
| CommandJobBuilder (platform/commands) | {deviceId, command, params} → Result<CommandDeviceJob, CommandResolutionError> — {{param}} çözümü + zorunlu param kontrolü + validate/atomic eşleme               | IDeviceConfigSource                                             |

**DI kuralları:** Tamamı constructor injection; config obje (ManagementServiceConfig, AlertNotifierConfig); arayüz I ön ekli; logger/notifier opsiyonel (yoksa console fallback + notify atlanır — device-service deseni).

## 4. Cycle Veri Toplama

**Sorun:** Cihazlar farklı poll aralıklarında okunur (1 sn / 5 sn, saniye ızgarasına hizalı — device-service.ts:204-209). Tek cycle'da tüm cihazların verisi aynı an üretilmez.

**Çözüm — snapshot birleştirme (kontrat değişikliği YOK):**

1. device-service her okumada zaten cihaz başına MANAGEMENT job yayınlar (değişmez).
2. CycleSnapshotStore.record(deviceId, telemetries) her job'ı kaydeder — (deviceId, name) → { value, recordedAt }.
3. Değerlendirme tick'inde snapshot() tüm cihazların en güncel değerlerini döner; recordedAt > now - maxAgeMs olmayan girişler bayat sayılır ve döndürülmez (kademeli bozulma — cihaz sessizse kural onu yok sayar, değerlendirmeyi durdurmaz).
4. maxAgeMs varsayılana: evaluationIntervalMs'in 3 katı (5 sn poll'lu cihaz 10 sn tick'te en fazla 1 tick bayat kalır; 3x tolerans).

**Neden yeni kuyruk/kontrat yok:** `queue_management` + mevcut job akışı yeterli; snapshot deseni `RealtimeSnapshotSource` ile aynı mantık.

## 5. Kural Modeli (Konfigürasyon Sözleşmesi)

Dosya: `services/management-service/deployment/config/rules.json` (mount: `/app/config/rules.json`).

```
{
  "rules": [
    {
      "name": "high-soc-stop",
      "enabled": true,
      "cooldownMs": 300000,
      "when": {
        "all": [
          {
            "device": { "types": ["bsc"] },
            "telemetry": "soc",
            "op": "gte",
            "threshold": 95,
            "debounceMs": 10000
          }
        ]
      },
      "then": [
        { "action": "command", "deviceId": "bsc-1", "command": "stop" },
        { "action": "log", "level": "warning", "eventCode":
"auto_rule_high_soc", "message": "BSC yüksek SoC – otomatik durdurma" },
        { "action": "notify" }
      ]
    }
  ]
}
```

Tipler (`packages/shared-types/src/automation-rule.ts`)

```
export type RuleOperator = "eq" | "neq" | "gt" | "gte" | "lt" | "lte";

export interface RuleDeviceSelector {
  ids?: string[]; // doğrudan cihaz kimlikleri
  types?: string[]; // cihaz tipleri (device config "type" alanından
çözömlenir)
}

export interface RuleCondition {
  device?: RuleDeviceSelector; // verilmezse: tüm cihazlar
  telemetry: string; // değer adı (ör. "soc")
  op: RuleOperator;
  threshold: number;
  debounceMs?: number; // koşulun kesintisiz sağlanması gereken süre
  (varsayılan 0)
```

```

}

export interface AutomationRuleWhen {
  all?: RuleCondition[];          // tümü aynı anda (en az 1 koşul – all veya
any zorunlu)
  any?: RuleCondition[];          // en az biri
}

export type RuleAction =
  | { action: "command"; deviceId: string; command: string; params?:
Record<string, unknown> }
  | { action: "log"; level: "info" | "warn" | "error"; eventCode?: string;
message?: string }
  | { action: "notify" };

export interface AutomationRule {
  name: string;                    // benzersiz; loglarda kural kimliği
  enabled?: boolean;               // varsayılan true
  cooldownMs?: number;             // aksiyon seti sonrası bastırma süresi
  (varsayılan 0)
  when: AutomationRuleWhen;
  then: RuleAction[];              // en az 1
}

export interface AutomationRulesFile {
  rules: AutomationRule[];
}

```

Zod: `automationRulesSchema` — bilinmeyen anahtarlar **strip** edilmez, reddedilir mi? Karar: tunnel operational config strip ediyordu; ama kural dosyası operatör hatası yakalamalı → **bilinmeyen anahtar hata** (`strict()`); geçersiz dosya = açılış reddi (fail-fast, §8).

## 6. Değerlendirme Semantiği (Davranış Sözleşmesi)

Her tick'te, her kural için:

- Hedef set çözümü:** koşul başına hedef cihazlar = `ids` u `types` çözümlenmiş set (`DeviceCatalog`); selector verilmemişse snapshot'taki tüm cihazlar.
- Koşul doğruluğu:** Bir koşul, hedef setteki **en az bir** cihazın ilgili `telemetry` değeri snapshot'ta mevcut + bayat değil + `op`'u sağlıyorsa TRUE. (Çok cihazda AND isteniyorsa iki ayrı koşul `when.all` altına yazılır — kompozisyon yeterli.)
- when birleşimi:** `all` → tüm koşullar TRUE; `any` → en az biri TRUE.
- Debounce:** Koşul `debounceMs` boyunca **kesintisiz** TRUE kalmalı — state: kural başına `heldSince` zamanı. `heldSince` set edildikten sonra her tick'te TRUE kalırsa, `now - heldSince >= debounceMs` olduğu anda koşul "aktif" sayılır.
- Kenar-tetik:** Kural yalnızca **inactive** → **active** geçişinde aksiyonları çalıştırır; aktif kaldığı sürece her tick'te TEKRARLAMAZ (AlarmTransitionDetector deseni — `device_alarm` yalnızca yükselen kenarda).
- Düşen kenar:** Koşul FALSE'a düşünce kural inactive'a döner ve `heldSince` sıfırlanır; yeniden yükselirse yeni oluşum sayılır.
- Cooldown:** Aksiyonlar çalıştıktan sonra `cooldownMs` dolmadan yeni yükselen kenar aksiyon tetiklemez. Cooldown `now()` enjeksiyonuyla deterministik test edilir.

8. **Tekrarlı tick güvenliği:** Değerlendirme saf sorgudur (state dışında yan etki yok); aynı snapshot ile iki kez çağrılırsa ikinci çağrı boş döner.

## 7. Aksiyonlar

| Aksiyon | Davranış                                                                                                                                                                                                                                  | Başarı logu                                                                         | Başarısızlık logu                                                                                                     |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| command | <code>CommandJobBuilder.build()</code><br>→ hata = <code>Result.err</code> →<br>aksiyon başarısız; job →<br><code>mq.executeAndWait(job, commandTimeoutMs)</code> (timeout =<br>komut timeoutMs + 2000 —<br>command-routes deseni)        | <code>auto_rule_action_ok</code><br>(info, context:<br>rule/deviceId/command/jobId) | <code>auto_rule_action_failed</code><br>(error, context:<br>rule/deviceId/command/reason)                             |
| log     | <code>TamperLogger.log(level, eventCode ?? "auto_rule_fired", message, context)</code> —<br>kategori app                                                                                                                                  | —                                                                                   | log hatası yutulmaz mı? Karar:<br>app kategorisi fail-closed<br>değildir; hata konsola düşer,<br>aksiyon akışı durmaz |
| notify  | <code>AlertNotifier.consider({ eventCode: "auto_rule_&lt;name&gt;", ... })</code> — notify sink hatası <b>yukarı fırlar</b> (AlertNotifier sözleşmesi) →<br>aksiyon başarısız sayılır,<br><code>auto_rule_action_failed</code><br>basılır | <code>auto_rule_action_ok</code>                                                    | <code>auto_rule_action_failed</code>                                                                                  |

- Aksiyonlar sırayla çalışır; biri başarısız olursa **sonrakiler çalışmaya devam eder** (kademeli bozulma — tüm sonuçlar loglanır).
- `command` aksiyonunun hedefi koşuldaki cihazdan **bağımsız** yazılabilir (ör. bsc-1 koşulu → cb-1'e komut); mantıksal bağ kural yazarının sorumluluğu.
- Logger/notifier enjekte edilmemişse: log aksiyonu console.warn'a düşer, notify aksiyonu atlanır (sonuç yine loglanır).

## 8. Hata Kategorileri

| Durum                                                     | Kategori                     | Davranış                                                                                          |
|-----------------------------------------------------------|------------------------------|---------------------------------------------------------------------------------------------------|
| <code>rules.json</code> okunamaz/geçersiz (zod)           | Beklenen —<br>açılış         | <b>Fail-fast:</b> <code>start()</code> reddedilir (servis yanlış kural dosyasıyla çalışmaz).      |
| Cihaz config'i yok / komut tanımsız / zorunlu param eksik | Beklenen —<br>çalışma zamanı | <code>CommandResolutionError (Result.err)</code> →<br>aksiyon başarısız logu; kural akışı durmaz. |
| <code>executeAndWait</code> timeout / job başarısız       | Beklenen —<br>çalışma zamanı | <code>JobResult.success=false</code> →<br><code>auto_rule_action_failed</code> .                  |
| Snapshot'ta veri yok / bayat                              | Beklenen —<br>veri durumu    | Koşul FALSE sayılır (bayat veriyle komut ÇALIŞTIRILMAZ — güvenlik).                               |

| Durum                             | Kategori          | Davranış                                                                             |
|-----------------------------------|-------------------|--------------------------------------------------------------------------------------|
| Aksiyon listesi boş / when boş    | Beklenen — config | zod reddeder (açılış fail-fast).                                                     |
| TamperLogger/AlertNotifier hatası | Kademeli          | Aksiyon sonucu <code>auto_rule_action_failed</code> ; sonraki aksiyonlar devam eder. |

## 9. Dağıtım ve Tier Wiring

- **Tier:** yalnızca **container** (cihazların poll edildiği stack). Field/boss MVP dışı.
- **Compose:** `docker-compose.container.dev.yml` + `docker-compose.container.yml`'e `management-service` eklenir.
- **Env (deployment/.env.container.example):**
  - `REDIS_HOST` / `REDIS_PORT` (ortak)
  - `DEVICE_CONFIG_DIR=/app/config-docker` (device-service ile aynı mount, `:ro`)
  - `MANAGEMENT_RULES_PATH=/app/config/rules.json` (kendi config mount'u, `:ro`)
  - `MANAGEMENT_EVALUATION_INTERVAL_MS` (varsayılan 10000)
- **Dockerfile:** `services/management-service/deployment/Dockerfile` + `Dockerfile.dev` (device-service deseni).
- **Nx:** `services/management-service/project.json` — `dev/test/typecheck`; `implicitDependencies`: `core`, `shared-types`, `platform-messaging`, `platform-commands`, `tamper-logger`.
- **Kuyruk kontratı:** `MANAGEMENT` job'ları **üretim tarafında değişmez** — device-service'a dokunulmaz.

## 10. Kabul Kriterleri

| Kod | Kriter                                                                                                                                                                                                   | Kanıt türü                                |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| K1  | <code>MANAGEMENT</code> job'ları snapshot'a kaydedilir; TTL bayat girişler değerlendirmeye girmez                                                                                                        | unit (CycleSnapshotStore)                 |
| K2  | Koşul inactive → active kenarında (debounce sonrası) aksiyonlar BİR kez çalışır; aktifken tekrarlanmaz                                                                                                   | unit (RuleEvaluator, fake timers)         |
| K3  | Cooldown içinde yeni yükselen kenar aksiyon tetiklemez; cooldown sonrası tetikler                                                                                                                        | unit (RuleEvaluator)                      |
| K4  | <code>command</code> aksiyonu <code>COMMAND_DEVICE</code> job üretir (config çözümleme + <code>{{param}}</code> + zorunlu param kontrolü) ve başarı/başarısızlığı ayrı <code>eventCode</code> 'la loglar | unit (CommandJobBuilder + ActionExecutor) |
| K5  | <code>notify</code> aksiyonu <code>AlertNotifier</code> üzerinden kural bazlı <code>eventCode</code> ile bildirim üretir                                                                                 | unit (ActionExecutor)                     |
| K6  | Geçersiz <code>rules.json</code> → açılış reddi (fail-fast); bilinmeyen anahtar reddedilir                                                                                                               | unit (RuleConfigLoader)                   |
| K7  | Örnek kural (bsc <code>soc &gt;= 95</code> → <code>stop</code> + log + notify) container dev stack'te gözle çalışır                                                                                      | gözle (compose)                           |
| K8  | Kapılar: yeni kod $\geq 70$ satır; <code>RuleEvaluator/ActionExecutor/CommandJobBuilder</code> $\geq 90$ branch                                                                                          | coverage raporu                           |

| Kod | Kriter                                                                                    | Kanıt türü                           |
|-----|-------------------------------------------------------------------------------------------|--------------------------------------|
| K9  | web-service komut akışı refactor sonrası mevcut testlerle yeşil kalır (davranış değişmez) | <code>nx run web-service:test</code> |

## 11. Görev Listesi (T1-T9)

| Görev | İçerik                                                                                                                                                                                                                                        |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| T1    | <code>packages/shared-types/src/automation-rule.ts</code> — tipler + zod <code>automationRulesSchema</code> + JSDoc kontratı (Aşama 2)                                                                                                        |
| T2    | <code>packages/platform/commands</code> — <code>IDeviceConfigSource</code> , <code>DeviceConfigFileSource</code> , <code>CommandJobBuilder</code> , <code>CommandResolutionError</code> ; <code>command-routes.ts</code> refactor (Aşama 2-4) |
| T3    | <code>services/management-service/src/cycle-snapshot-store.ts</code>                                                                                                                                                                          |
| T4    | <code>services/management-service/src/rule-evaluator.ts</code>                                                                                                                                                                                |
| T5    | <code>services/management-service/src/action-executor.ts</code>                                                                                                                                                                               |
| T6    | <code>services/management-service/src/rule-config-loader.ts</code> + <code>device-catalog.ts</code>                                                                                                                                           |
| T7    | <code>management-service.ts</code> (wiring/yaşam döngüsü) + <code>run.ts</code> + <code>package.json/project.json/vitest.config.ts</code> + Dockerfile'lar                                                                                    |
| T8    | Compose (container dev+prod) + <code>.env.container.example</code> + <code>config/rules.example.json</code>                                                                                                                                   |
| T9    | Aşama 5-6 dokümanları: <code>MANAGEMENT-SERVICE-DOGRULAMA.md</code> , <code>MANAGEMENT-SERVICE-TEST-KAPSAMI.md</code> + <code>test-envanteri.md</code> güncelleme                                                                             |

## 12. Aşama Eşlemesi (6 aşamalı iş akışı)

| Aşama                | Ürün                                                                                                                                                                                                                                                                                     |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. SPEC              | Bu doküman                                                                                                                                                                                                                                                                               |
| 2. JSDoc             | <code>automation-rule.ts</code> , <code>commands</code> paketi, servis sınıfları (her T görevinde)                                                                                                                                                                                       |
| 3. TEST<br>(kırmızı) | <code>cycle-snapshot-store.test.ts</code> , <code>rule-evaluator.test.ts</code> , <code>action-executor.test.ts</code> , <code>rule-config-loader.test.ts</code> , <code>device-catalog.test.ts</code> , <code>command-job-builder.test.ts</code> + web-service karakterizasyon koruması |
| 4. IMPL<br>(yeşil)   | T1-T8                                                                                                                                                                                                                                                                                    |
| 5. SONUÇ             | <code>MANAGEMENT-SERVICE-DOGRULAMA.md</code>                                                                                                                                                                                                                                             |
| 6. KAPSAM            | <code>MANAGEMENT-SERVICE-TEST-KAPSAMI.md</code> + <code>test-envanteri.md</code>                                                                                                                                                                                                         |